



ENGINEERING INTERNSHIP REPORT

*Design and Implementation of Real-Time Embedded Systems
Based on FPGA Using the Zybo Z7-20 Platform*

Host Organization: CRTI (Industrial Technologies Research Center)

Department: Embedded Systems Division

Supervisor: Amina Skander (Researcher, Engineer)

Intern: Boughendja Yesser Zakaria

Internship Period: 21/04/2026 → 10/06/2026

ACKNOWLEDGEMENTS:

I would like to express my sincerest gratitude for the internship opportunity granted to me at the Research Center in Industrial Technologies (CRTI).

I extend my special thanks to my supervisor, Ms. SKANDER Amina, for her dedication, availability, and guidance throughout my internship. Her constant support and invaluable advice have enabled me to progress significantly, both technically and professionally. Thanks to her mentorship, I was able to thrive in a stimulating, enriching, and learning-conducive environment. I particularly appreciated her pedagogical approach, which seamlessly combined rigor with encouragement, allowing me to fully realize my potential.

This internship has been an incredibly rewarding professional experience for me. The warm welcome, the quality of supervision, and the trust placed in me allowed me to considerably advance my technical and professional skills. Thanks to your support and the expertise of your team members, I was not only able to apply my theoretical knowledge in practice but also discover new methodologies and approaches in the field of industrial technologies.

The collaborative and stimulating work environment within your center enabled me to foster a strong professional drive and gain a practical understanding of the challenges and solutions in industrial research. I especially appreciated the opportunity to work on diverse projects and actively contribute to the department's objectives. I would also like to express my gratitude to the entire CRTI team for their availability, kindness, and valuable insights, which facilitated my integration and enriched my experience.

Finally, I would like to thank my academic institution for making this experience possible and for its continuous support throughout my academic journey. This internship will remain a milestone in my education, leaving me enriched not only professionally but personally as well. I hope to stay in touch and look forward to the possibility of future collaborations.

Table of content:

Chapter 1: General Introduction

- 1.1 Context of embedded systems
- 1.2 Importance of FPGAs in modern engineering systems
- 1.3 Objectives of the internship
- 1.4 Overview of the Zybo Z7-20 platform

Chapter 2: CRTI Presentation and Technical Environment

- 2.1 Overview of CRTI
- 2.2 Embedded systems research division

Chapter 3: FPGA and Zynq Architecture

- 3.1 FPGA fundamentals
- 3.2 Zynq architecture (Processing System / Programmable Logic)
- 3.3 VHDL description language
- 3.4 Vivado design flow
- 3.5 Zybo Z7-20 hardware platform

Chapter 4: Programmable Logic (FPGA) Implementations

- 4.1 Basic input/output systems
- 4.2 Counters, timers, and decoders
- 4.3 4-bit arithmetic system (ALU design inspired)
- 4.4 Mode-based control systems

Chapter 5: Embedded Hardware Interfacing Systems

- 5.1 PWM control and servo motor
- 5.2 Ultrasonic distance measurement system
- 5.3 LCD display integration
- 5.4 Embedded radar scanning system

Chapter 6: Video Stream Integration

- 6.1 Camera acquisition system
- 6.2 Real-time video display pipeline

Chapter 7: Technical Analysis and Engineering Feedback

7.1 FPGA design complexity

7.2 Hardware-level debugging challenges

7.3 Comparison with microcontroller systems

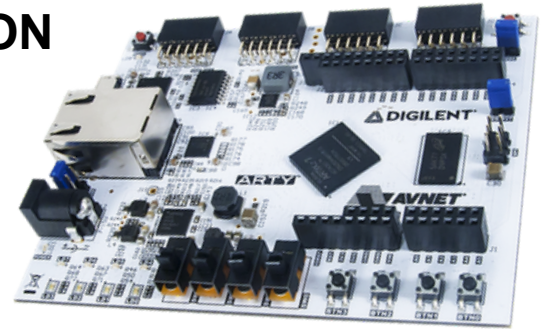
7.4 Skills and engineering insights gained

Chapter 8: General Conclusion

CHAPTER 1: GENERAL INTRODUCTION

1.1 Context of Embedded Systems

Embedded systems are computing systems designed to perform dedicated functions within larger technical or industrial environments. Unlike general-purpose computers, they are tightly constrained by requirements such as real-time execution, power efficiency, deterministic behavior, and hardware integration.



In modern engineering applications, embedded systems are no longer limited to simple control tasks. They now form the core of complex systems such as autonomous robotics, industrial automation, aerospace control systems, and real-time signal processing platforms. This evolution is largely driven by the increasing demand for systems that combine computation, sensing, and actuation within a unified architecture.

Within this context, FPGA-based systems and heterogeneous SoC platforms have emerged as critical technologies, enabling parallel hardware execution and deterministic timing that cannot be achieved using classical microcontroller architectures alone.

1.2 Importance of FPGAs in Modern Engineering Systems

Field Programmable Gate Arrays (FPGAs) are reconfigurable hardware devices that allow the implementation of digital circuits directly at the hardware level. Unlike microprocessors, which execute sequential instructions, FPGAs enable true parallel execution of logic circuits, making them particularly suitable for:

- Real-time signal processing
- High-speed data acquisition
- Control systems requiring deterministic timing
- Hardware acceleration of computational tasks

The main advantage of FPGA technology lies in its ability to implement custom digital architectures tailored to specific applications. However, this flexibility comes at the...

...cost of increased design complexity, requiring hardware-level thinking and a strong understanding of digital logic design principles.

1.3 Objectives of the Internship

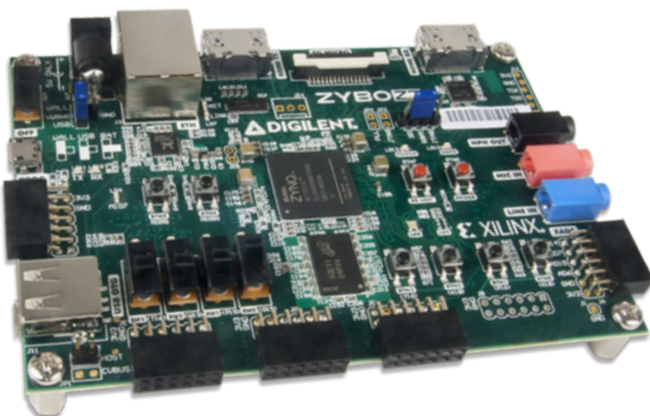
The main objective of this internship was to gain practical experience in the design, implementation, and integration of embedded systems based on FPGA and SoC architectures.

More specifically, the internship aimed to:

- Understand the architecture and functioning of FPGA-based systems
- Learn hardware description using VHDL
- Use the Vivado design environment for synthesis and implementation
- Develop and test digital logic modules on FPGA hardware
- Interface sensors and actuators with embedded systems
- Implement real-time control applications using both programmable logic (FPGA) and processing systems (ARM cores)

A secondary objective was to develop engineering autonomy by working in a research-oriented environment, where problem-solving, debugging, and system integration were performed with minimal direct supervision.

1.4 Overview of the Zybo Z7-20 Platform



The Zybo Z7-20 is a development board based on the Xilinx Zynq-7000 SoC architecture. It integrates two main subsystems:

- Processing System (PS): Dual-core ARM Cortex-A9 processor used for software execution
- Programmable Logic (PL): FPGA fabric used for hardware implementation of digital circuits

This hybrid architecture allows tight coupling between software and hardware components, enabling efficient implementation of embedded systems that require both computational flexibility and hardware-level performance.

The Zybo Z7-20 provides multiple interfaces that are essential for embedded development, including:

- GPIO for digital input/output
- PWM outputs for motor control
- HDMI and camera interfaces
- Audio and communication peripherals
- Expandable headers for external sensors

In this internship, the platform was used as a complete embedded development environment for implementing digital logic systems, sensor interfacing, actuator control, and real-time system integration.

CHAPTER 2: CRTI PRESENTATION AND TECHNICAL ENVIRONMENT

2.1 Overview of CRTI (Industrial Technologies Research Center)



The Industrial Technologies Research Center (CRTI) is a national research institution in Algeria focused on applied research and technological development in industrial engineering domains. Its mission includes supporting innovation, developing prototype systems, and contributing to technology transfer between academic research and industrial applications.

CRTI operates across multiple engineering domains, including

automation, embedded systems, industrial control, materials engineering, and digital technologies. The organization is structured into specialized research divisions, each dedicated to a specific technical field.

Within this framework, the Embedded Systems Division plays a central role in the development of intelligent electronic systems, real-time control platforms, and hardware-software integrated architectures.

2.2 Embedded Systems Research Division

The Embedded Systems Division is dedicated to the design and implementation of hardware and software solutions for real-time applications. Its work typically involves:

- Digital system design using FPGA and ASIC methodologies
- Embedded software development for microcontrollers and SoCs
- Sensor integration and signal acquisition systems

- Actuator control systems for robotics and automation
- Research-oriented prototyping and experimental validation

Projects within this division are generally characterized by a strong emphasis on hardware-level implementation, system integration, and experimental validation rather than purely theoretical modeling.

During the internship, the working environment was primarily research-oriented, with a focus on independent learning, system experimentation, and iterative debugging. Technical supervision was available, but the majority of the development process relied on autonomous problem-solving and self-directed progress.

2.3 Internship Organization and Working Environment

The internship was conducted over a period of approximately 9 weeks corresponding to an estimated workload of around 200 hours. The Working structure was flexible and task oriented rather than strictly scheduled.

The development process followed an incremental methodology:

- Initial familiarization with FPGA development tools and hardware environment
- Implementation of basic digital logic systems (LEDs, switches, buttons)
- Gradual integration of sequential logic modules (timers, counters)
- Expansion toward more complex systems involving sensors and actuators
- Final integration of multi-component embedded systems

The intern operated with a high degree of autonomy (approximately 90%), performing tasks such as:

- FPGA design and VHDL implementation
- Hardware debugging and functional verification



- Sensor and actuator interfacing
- System level integration on the Zybo Z7-20 platform

Interactions with other interns and researchers provided additional technical insight, particularly regarding ongoing projects and system-level design approaches within the laboratory.

This working environment encouraged independent engineering development, reinforcing practical skills in debugging, hardware reasoning, and system integration under real constraints.

2.4 Development Toolchain and Technical Environment

The technical environment used during the internship was centered around the Xilinx FPGA development ecosystem:

Vivado Design Suite:

Vivado was the primary tool used for FPGA development.

It supports the full hardware design flow, including:

- HDL design entry (VHDL in this case)
- Synthesis of digital logic
- Implementation (mapping, placement, routing)
- Bitstream generation for FPGA programming

Vivado also provides IP integration tools, allowing the inclusion of pre-designed hardware modules into custom systems.



Xilinx SDK / Vitis (Processing System Programming)

The Processing System (PS) of the Zynq architecture was programmed using the Xilinx SDK environment. This allowed C-based development targeting the ARM Cortex-A9 processor.

This layer was used for:

- High-level system control
- Peripheral configuration
- Interaction with FPGA-based hardware modules
- Embedded software execution



Hardware Environment

The hardware setup included:

- Zybo Z7-20 FPGA development board
- Ultrasonic distance sensors
- Servo motors with PWM control
- LCD display modules
- Camera module for video acquisition
- Level shifters and ADC-related interfacing components
- General-purpose electronic components for prototyping

2.5 FPGA vs Microcontroller Architectures:

A key conceptual difference explored during the internship is the distinction between microcontroller-based systems and FPGA-based systems:

Microcontroller Systems

Microcontrollers execute instructions sequentially on a fixed processor architecture. They are efficient for general-purpose embedded applications but are limited in:

- Parallel execution capability
- Timing determinism under heavy computational load
- Custom hardware acceleration



FPGA Systems

FPGAs implement logic directly in hardware. Each function is realized as a physical digital circuit rather than software instructions. This enables:

- True parallel execution of operations
- Precise timing control (deterministic behavior)
- Custom hardware architectures tailored to the application

However, FPGA development introduces additional complexity, particularly in:

- Hardware-level debugging
- Timing constraints and synchronization
- Increased design and verification effort

Zynq Hybrid Architecture:

The Zybo Z7-20 platform combines both approaches through the Zynq SoC architecture:

Processing System (PS): software execution (ARM cores)

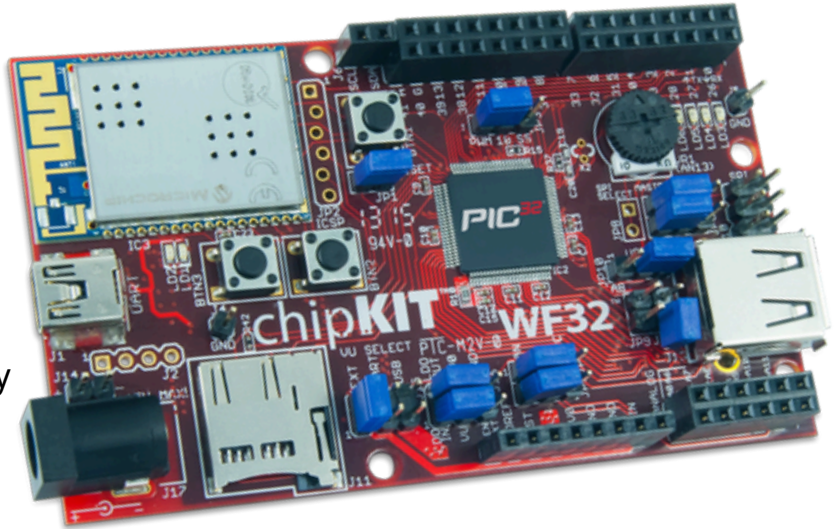
Programmable Logic (PL): hardware acceleration (FPGA fabric)

This hybrid model allows efficient partitioning of tasks between software and hardware, optimizing both flexibility and performance.

CHAPTER 3:FPGA AND ZYNQ ARCHITECTURE

3.1 FPGA Fundamentals:

A Field Programmable Gate Array (FPGA) is a reconfigurable integrated circuit composed of a large array of programmable logic blocks interconnected through a configurable routing network. Unlike traditional processors, which execute instructions sequentially an FPGA implements digital circuits directly in hardware.



Each logic block typically contains configurable lookup tables (LUTs), flip-flops, and multiplexers, allowing the implementation of combinational and sequential logic. By configuring these elements, the designer effectively defines a custom digital hardware architecture.

The key characteristic of FPGA systems is parallelism at the hardware level. Multiple operations can execute simultaneously, provided they are mapped to independent logic resources. This makes FPGAs particularly suitable for real-time and high-throughput applications.

However, FPGA design requires strict attention to timing constraints, clock domains, and synchronous design principles. Unlike software, behavior is determined by physical signal propagation and clocked hardware structures.

3.2 Zynq Architecture (Processing System / Programmable Logic)

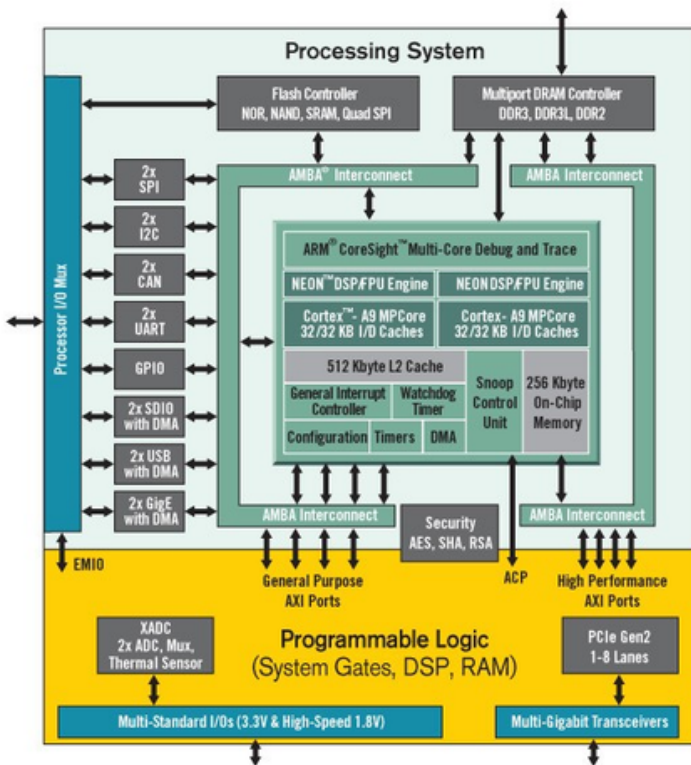
The Xilinx Zynq-7000 architecture, used in the Zybo Z7-20 platform, integrates two heterogeneous subsystems:

Processing System (PS).

The Processing System consists of a dual-core ARM Cortex-A9 processor responsible for executing software applications. It includes:

- CPU cores for general-purpose computation
- Memory controllers (DDR interface)
- Standard peripherals (UART, SPI, I2C, etc.)
- Operating system or bare-metal execution support

The PS is used for high-level control logic and software-based decision-making.



Programmable Logic (PL)

The Programmable Logic corresponds to the FPGA fabric. It is used to implement custom digital hardware circuits described using hardware description languages such as VHDL or Verilog.

The PL is responsible for:

- Real-time digital signal processing
- Custom hardware accelerators
- Direct interfacing with external hardware (sensors, LEDs, motors)
- Time-critical control systems

PS/PL Communication

The PS and PL communicate through high-speed AXI (Advanced eXtensible Interface) buses. This allows:

- Memory-mapped communication between software and hardware
- Low-latency control of FPGA modules from the ARM processor
- Efficient hardware/software co-design

This separation enables a hybrid design methodology where computational tasks are distributed between software and hardware depending on performance requirements.

3.3 VHDL Hardware Description Language

VHDL (VHSIC Hardware Description Language) is a formal language used to describe digital hardware systems.

Unlike programming languages such as C, VHDL does not describe sequential execution.

Instead, it defines concurrent hardware behavior, where all described processes exist in parallel. A VHDL design typically consists of:

- Entity: Defines the interface (inputs and outputs)
- Architecture: Defines the internal behavior or structure

VHDL supports two main modeling styles:

- Combinational logic: output depends only on current inputs
- Sequential logic: output depends on inputs and internal state (clocked systems)

In this internship, VHDL was used to implement counters, timers, arithmetic units, and control logic systems, all operating synchronously with a system clock. A key concept is that VHDL describes hardware that is synthesized into physical circuits, meaning design errors directly translate into hardware-level malfunction rather than runtime software errors.

3.4 Vivado Design Flow

The Xilinx Vivado Design Suite provides the complete toolchain for FPGA development. The design flow followed during the internship consists of the following stages:

1. Design Entry

Digital circuits are described using VHDL. Designer defines entities, architectures, and signal behavior.

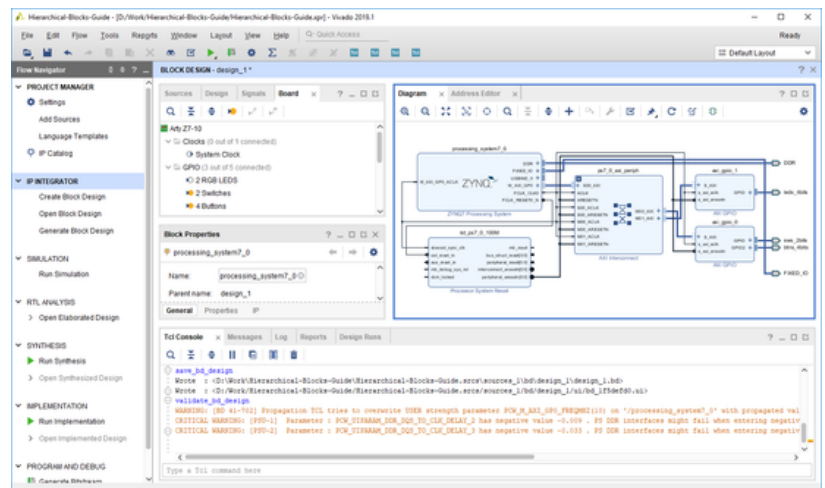
2. Synthesis

Vivado converts the VHDL code into a gate level representation composed of logic primitives (LUTs, flip-flops, etc.). This stage verifies logical correctness.

3. Implementation

The synthesized design is mapped onto the physical FPGA resources. This stage includes:

- Placement of logic elements
- Routing of interconnections
- Timing optimization



Timing constraints are applied to ensure correct synchronization with the system clock.

4. Bitstream Generation

Once implementation is complete, Vivado generates a bitstream file (.bit), which represents the final configuration of the FPGA hardware.

5. Programming

The bitstream is uploaded to the Zybo Z7-20 board, configuring the FPGA fabric to behave as the designed digital system.

3.5 Zybo Z7-20 Hardware Platform

The Zybo Z7-20 is a development board based on the Xilinx Zynq-7000 SoC. It provides a complete embedded systems platform combining processing and programmable logic capabilities.

Key Hardware Features:

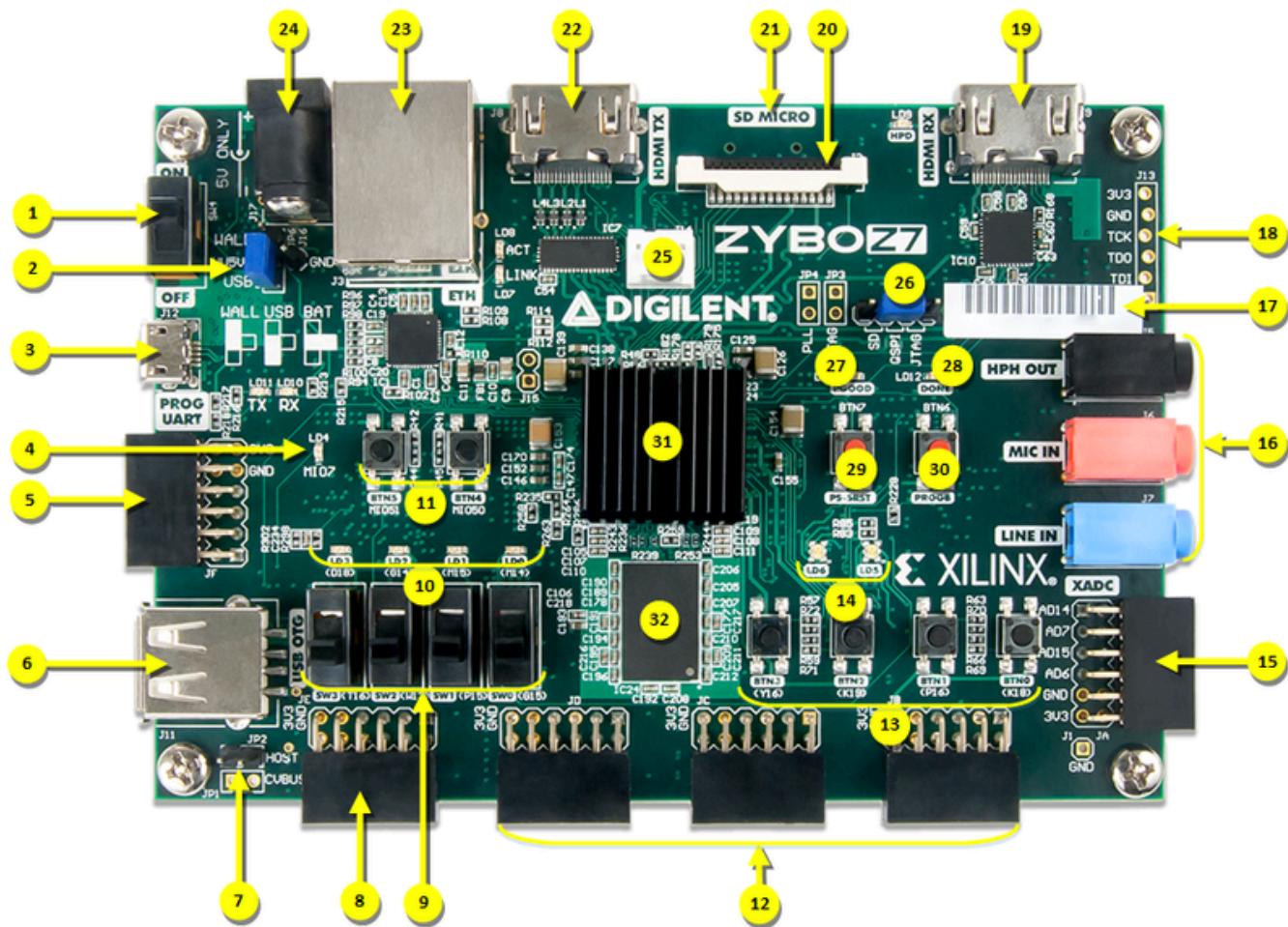
- Zynq-7020 SoC (dual-core ARM Cortex-A9 + FPGA fabric)
- GPIO interfaces for LEDs, switches, and buttons
- PWM-capable outputs for motor control
- USB and Ethernet connectivity
- HDMI and camera interfaces (depending on configuration)
- Expansion headers for external sensors and modules
-

Role in This Internship:

The Zybo Z7-20 served as the central platform for all implemented systems. It enabled:

- Direct FPGA-based digital logic design
- Real-time interfacing with sensors (ultrasonic module)
- Actuator control (servo motor via PWM)
- Integration with external display systems (LCD)
- Video stream acquisition experiments

This platform allowed the practical implementation of a complete embedded system pipeline, from low-level digital logic to real-world hardware interaction.



Callout	Description	Callout	Description	Callout	Description
1	Power Switch	12	High-speed Pmod ports *	23	Ethernet port
2	Power select jumper	13	User buttons	24	External power supply connector
3	USB JTAG/UART port	14	User RGB LEDs *	25	Fan connector (5V, three-wire) *
4	MIO User LED	15	XADC Pmod port	26	Programming mode select jumper
5	MIO Pmod port	16	Audio codec ports	27	Power supply good LED
6	USB 2.0 Host/OTG port	17	Unique MAC address label	28	FPGA programming done LED
7	USB Host power enable jumper	18	External JTAG port	29	Processor reset button
8	Standard Pmod port	19	HDMI input port	30	FPGA clear configuration button
9	User switches	20	Pcam MIPI CSI-2 port	31	Zynq-7000
10	User LEDs	21	microSD connector (other side)	32	DDR3L Memory
11	MIO User buttons	22	HDMI output port	* denotes difference between Z7-10 and Z7-20	

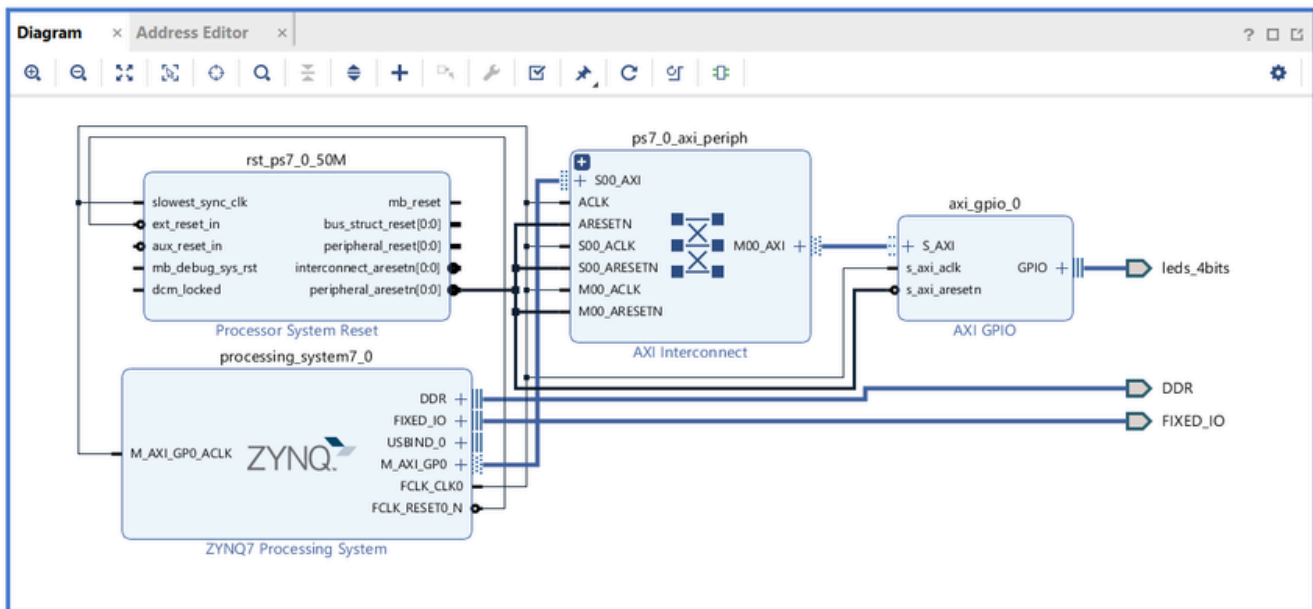
CHAPTER 4: PROGRAMMABLE LOGIC (FPGA IMPLEMENTATIONS)

4.1 Basic Input/Output Systems

The first stage of development on the Programmable Logic (PL) consisted of implementing basic input/output (I/O) systems using the Zybo Z7-20 GPIO resources. These systems served as an initial validation of the FPGA toolchain, hardware programming flow, and synchronous digital design methodology.

LED Control:

LEDs were used as primary output indicators. Simple combinational and sequential logic was implemented in VHDL to control LED states based on internal signals or external inputs.



Typical operations included:

- Direct mapping of switches to LEDs
- Conditional LED activation based on button presses
- Clock-driven LED toggling for behavioral verification
- Switches and Buttons

Switches were used as static input signals, while buttons provided dynamic input events. The distinction between level-triggered (switches) and edge-sensitive (buttons) inputs was essential for designing stable synchronous circuits.

Debouncing considerations were implicitly handled through synchronous sampling using the system clock.

4.2 Sequential Logic Systems: Counters, Timers, and Decounters

Sequential systems formed the core of the FPGA learning process, as they introduce state retention and clock dependency.

Binary Counter (0 to 15):

A 4-bit counter was implemented using flip-flop-based state storage. The system increments its state on each rising clock edge, producing a binary sequence from 0000 to 1111.

This design demonstrated:

- Clock-driven state evolution
- Binary representation of integer values
- Modular arithmetic behavior (wrap around at overflow)
- Up/Down Counter (Increment Decrement System)

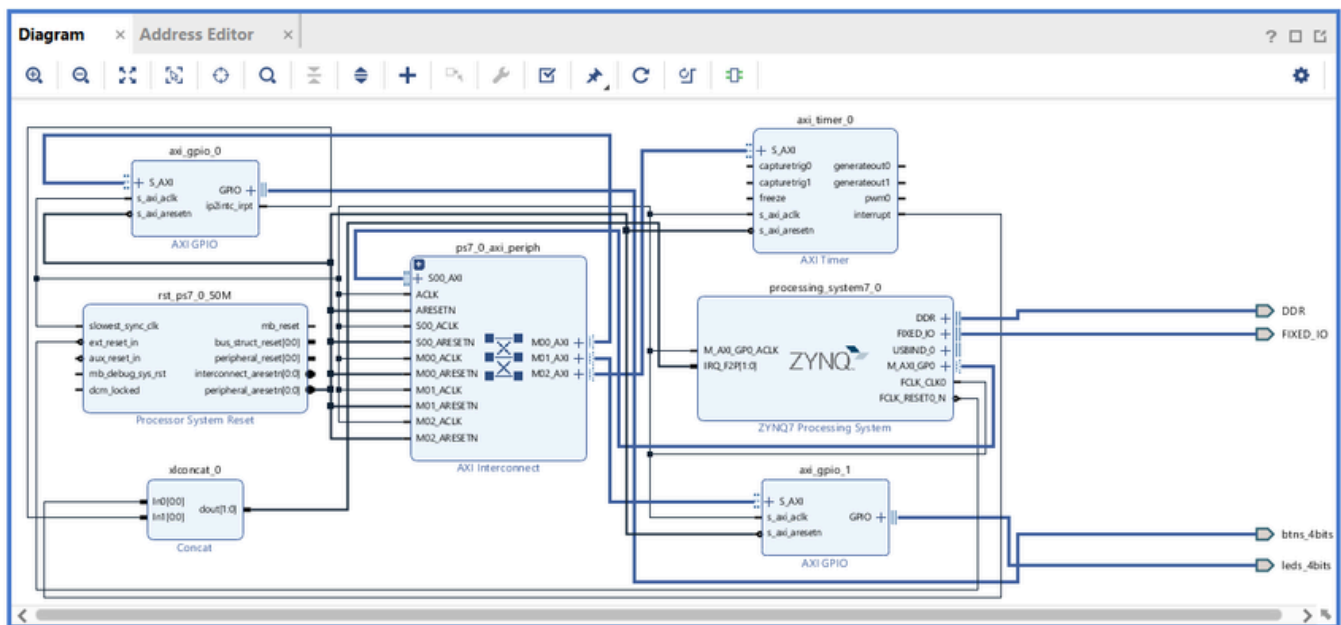


A bidirectional counter was designed using switch-controlled mode selection. Depending on the selected mode, the system either increments or decrements the stored value.

This introduced:

- Conditional next-state logic
- Mode-based state transitions
- Control-path separation from data-path logic
- Timer / Decounter System

A timer-like behavior was implemented by reversing the counting direction and introducing reset and freeze modes.



The system included:

- Reset-to-zero initialization
- Decrement mode acting as a countdown
- Freeze mode preserving the current state without clock updates

This required careful synchronous control to avoid unintended state transitions during mode switching.

4.3 4-bit Arithmetic System (ALU-like Design)

A simplified arithmetic logic unit (ALU) was implemented using a 4-bit architecture capable of performing basic arithmetic operations.

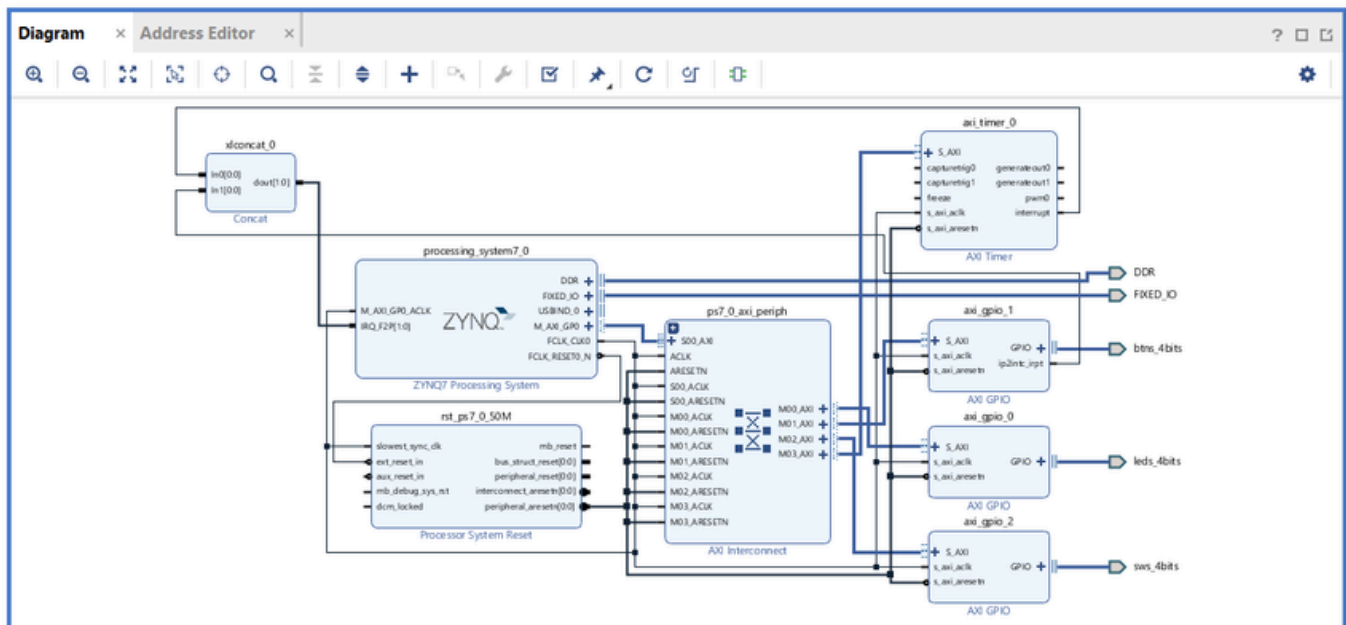
The system operates on two 4-bit operands and supports: (Addition, subtraction and result storage and display) where inputs were controlled via switches:

- First operand selection and storage
- Second operand selection
- Operation selection (addition/subtraction)
- Reset functionality

This module introduced several important hardware design concepts:

- Register-based data storage
- Combinational arithmetic operations
- Control logic for operand selection
- Overflow handling within 4-bit constraints (0–15 range)

The system behavior is fully deterministic and depends on synchronous updates triggered by the system clock.



4.4 Mode-Controlled System Architecture

To increase system flexibility, multiple functional behaviors were integrated into a single unified architecture using mode selection via switches.

Supported Modes

- Reset mode: forces system state to zero
- Increment mode: counter increases over time
- Decrement mode: counter decreases over time
- Freeze mode: system state is held constant
-

This architecture is based on a finite state control approach, where:

- Input switches define the active mode
- A central control unit selects the next state logic
- Only one behavior is active at a time

This design reflects a key FPGA principle: hardware reuse through reconfigurable logic behavior rather than software branching.

4.5 Summary of Programmable Logic Work

The programmable logic development phase established the foundational understanding of FPGA design principles, including:

- Synchronous sequential logic design
- State machine behavior
- Binary arithmetic implementation
- Mode-based hardware control systems

These implementations served as the basis for more advanced system integration involving sensors, actuators, and real-time embedded interfaces in subsequent chapters.

CHAPTER 5: EMBEDDED HARDWARE INTERFACING SYSTEMS

5.1 Servo Motor Control Using PWM

Pulse Width Modulation (PWM) is a fundamental technique used to control the position of servo motors. A servo motor interprets the width of a periodic pulse signal to determine its angular position. A standard servo motor expects a control signal with a fixed frequency (typically ~50 Hz) and a variable pulse width (typically 1 ms to 2 ms within a 20 ms period). The angular position is directly related to the duty cycle of the PWM signal where short pulses implement smaller angle and long pulse for larger angle while it maintains its centered position with mid pulses.

In this internship, PWM signals were generated using FPGA logic by:

- Creating a clock-based counter
- Comparing the counter value with a configurable threshold
- Generating a high signal when the counter is below the threshold

This approach ensures precise timing control, which is critical for stable servo positioning. Unlike software PWM (microcontrollers), FPGA-based PWM offers:

- Deterministic timing behavior
- Reduced jitter
- Parallel generation of multiple control signals

5.2 Ultrasonic Distance Measurement System

The ultrasonic sensor system was implemented for real-time distance estimation using sound wave propagation. the system operates using two signals:

Trigger pulse: initiates ultrasonic emission

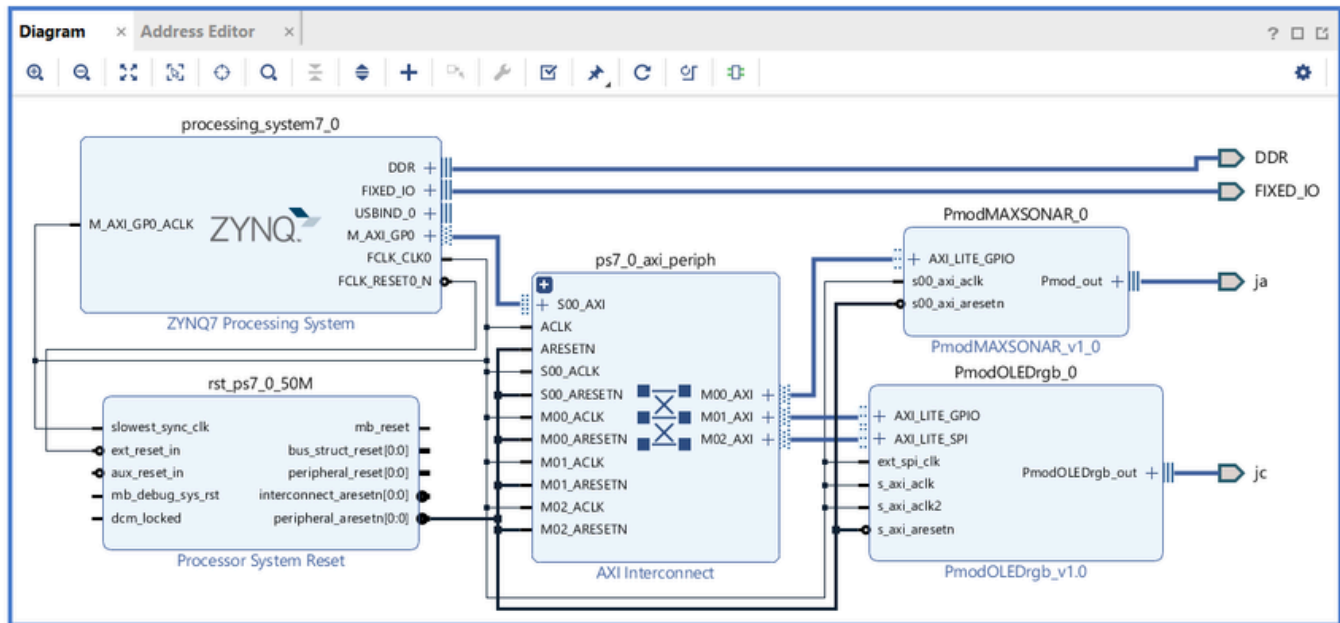
Echo signal: returns after reflection from an object

The distance is computed based on the time-of-flight of the ultrasonic wave. the measured time delay corresponds to the round-trip travel of the wave:

- The signal travels to the object and back
- Distance is proportional to the measured time interval

This requires precise timing measurement, which was implemented using FPGA counters. The system includes:

- Trigger signal generation
- Echo pulse duration measurement using a high-resolution counter
- Conversion of time values into distance representation
- This design highlights one of the main advantages of FPGA systems: accurate real-time timing measurement without software latency.



5.3 LCD Display Integration

The LCD module was used to display real-time system information such as measured distance and servo angle. The display system provides:

- Numerical representation of sensor data
- Real-time feedback for debugging and validation
- Visualization of system state (angle, distance, mode)

The LCD was interfaced using a structured communication protocol, where:

- Data signals were synchronized with control signals
- Timing constraints were respected for stable display updates
- Data formatting was handled before transmission to the display module

5.4 Rotating Radar System (Servo + Ultrasonic Sensor)

This system represents one of the most complete embedded integrations developed during the internship. The radar-like system combines:

- A servo motor for angular rotation
- An ultrasonic sensor mounted on the servo
- FPGA logic for measurement and control
- LCD display for real-time visualization
- Working Principle

The system performs a scanning operation:

- Servo rotates to a specific angle
- Ultrasonic sensor measures distance at that angle
- Data is recorded and displayed
- System increments angle and repeats

This creates a 2D scanning effect similar to basic radar systems.

Engineering Extension: Geometric Distance Projection:

In advanced operation, the system computes additional geometric information:

- Measured distance (radial component)
- Angular position from servo
- Projection of distance onto axes using trigonometric decomposition

This allows estimation of:

- Horizontal distance
- Perpendicular distance components
-

System Characteristics

- Real-time operation
- Sequential scanning behavior
- Multi-module synchronization (PWM + sensor + display)
- Embedded data fusion between hardware subsystems

5.5 System Integration Challenges:

Integrating sensors and actuators on FPGA introduces several engineering constraints:

- Strict timing synchronization between modules
- Signal noise and stability in physical interfaces
- Clock domain consistency
- Hardware-level debugging complexity

Unlike software systems, errors in FPGA design often originate from architectural issues rather than runtime exceptions, making debugging significantly more demanding.

CHAPTER 6: VIDEO STREAM INTEGRATION

6.1 Camera Acquisition System

As an extension of the embedded system integration work, a basic camera acquisition pipeline was implemented to handle real-time video input on the FPGA-based platform. The objective of this stage was not advanced image processing, but rather:

- Capturing a live video stream
- Transferring raw image data into the embedded system pipeline
- Displaying the stream in real time on an external screen or host device

This allowed evaluation of the FPGA system's capability to handle continuous high-throughput data streams. The camera system operates through a simple acquisition chain:

- Optical sensor captures frames
- Raw pixel data is transmitted through a communication interface
- FPGA or processing system receives the stream
- Data is forwarded for display output

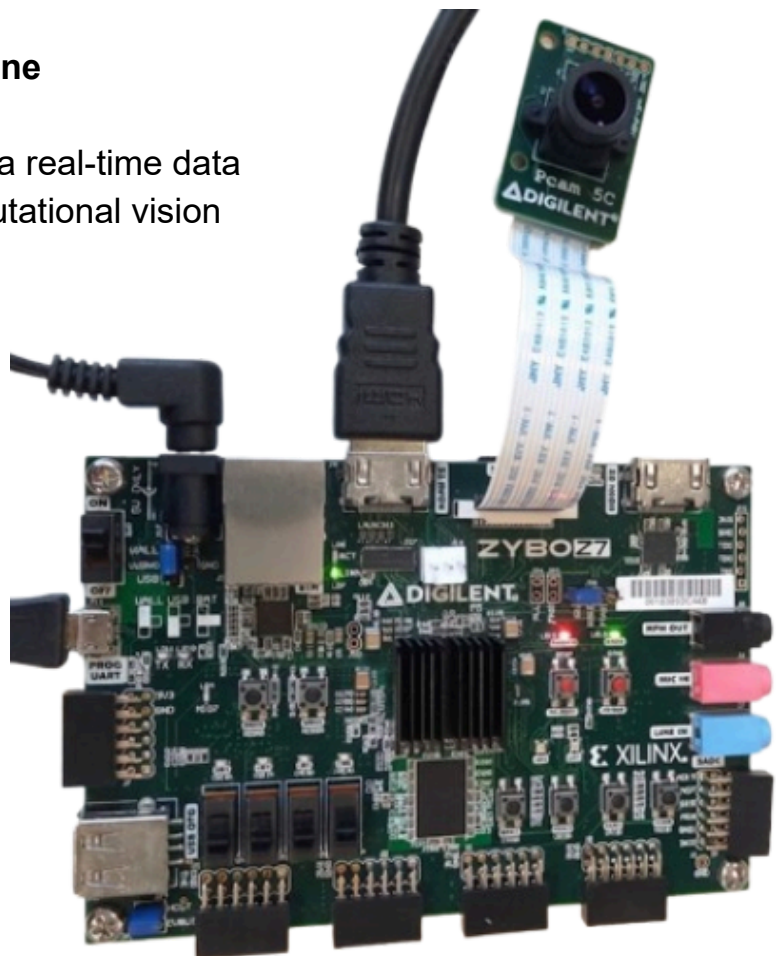
6.2 Real-Time Video Display Pipeline

The video pipeline was designed as a real-time data handling system rather than a computational vision system. The pipeline ensures:

- Continuous frame acquisition without interruption
- Minimal buffering delays to preserve real-time behavior
- Direct forwarding of video frames to output display

From a system design point of view, this stage primarily demonstrates:

- High-throughput data handling capability
- Streaming architecture principles



6.3 System-Level Integration Considerations

Integrating video streams into an FPGA-based embedded system introduces significantly higher constraints compared to discrete sensor systems. The key Constraints:

- High data rate requirements compared to sensors like ultrasonic modules
- Strict timing synchronization between input and output pipelines
- Memory bandwidth limitations for buffering frames
- Interface compatibility between camera module and processing system

This stage highlighted the difference between:

- Event-driven systems (e.g., buttons, sensors, PWM control)
- Continuous streaming systems (e.g., video acquisition)
-

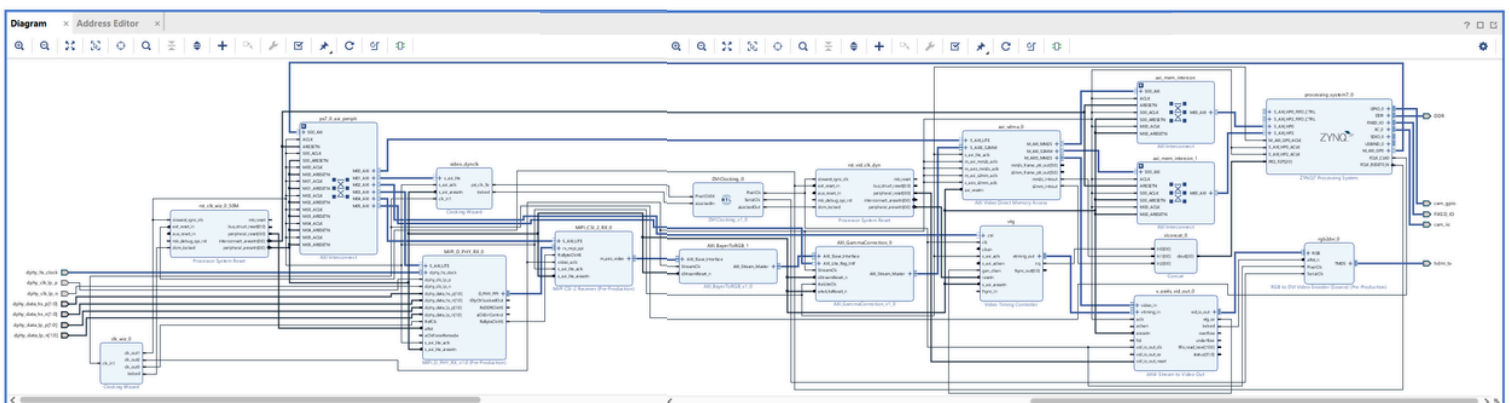
Video processing introduces continuous data flow, requiring pipeline-oriented thinking rather than discrete state transitions.

6.4 Summary of Video Integration Work

The camera integration task demonstrated the ability of the system to handle:

- Continuous real-time data streams
- Hardware–software coordination for data handling
- Basic video acquisition and display pipeline construction

Although no advanced image processing algorithms were implemented, this stage reinforced key embedded systems concepts such as throughput management, buffering, and real-time data synchronization.



CHAPTER 7: TECHNICAL ANALYSIS AND ENGINEERING FEEDBACK

7.1 FPGA Design Complexity and Learning Curve

Working with FPGA-based systems introduces a fundamentally different engineering paradigm compared to traditional software or microcontroller development. The main difficulty does not lie in syntax or tool usage, but in the conceptual shift toward hardware-level thinking. Unlike software execution, where instructions are sequential and abstracted from physical implementation, FPGA design requires the engineer to think in terms of:

- concurrent hardware structures
- signal propagation and timing
- synchronous state evolution
- physical resource mapping (LUTs, flip-flops, routing paths)

This results in a significantly steeper learning curve, especially during the initial stages of design where small conceptual errors can lead to non-functional hardware behavior after synthesis.

7.2 Hardware-Level Debugging Challenges

Debugging FPGA systems is inherently more complex than debugging software systems due to the absence of traditional runtime error feedback.

In this internship, debugging typically involved:

- verifying signal behavior through simulation and hardware observation
- checking timing consistency across clocked modules
- validating correct synthesis of VHDL descriptions
- analyzing unexpected hardware outputs on LEDs, displays, or sensors

A major challenge is that many errors only become visible after synthesis and implementation, meaning that:

- logical correctness does not guarantee hardware correctness
- small design mistakes can propagate into structural hardware faults
- debugging requires iterative redesign rather than simple code correction

This reinforces the need for careful architectural planning before implementation.

7.3 Timing Constraints and Deterministic Behavior

One of the most critical aspects of FPGA design is timing control. All sequential logic depends on a global or local clock signal, and system correctness depends on strict synchronization between modules. The key timing-related considerations include:

- clock edge alignment for state transitions
- propagation delay across combinational paths
- setup and hold time constraints
- avoiding race conditions between signals

In this internship, systems such as counters, PWM generation, and ultrasonic measurement required precise timing behavior. Any deviation in timing could result in incorrect counts, unstable motor control, or inaccurate distance measurements. This highlights one of the main strengths of FPGA systems: deterministic real-time behavior, provided that timing constraints are correctly respected.

7.4 System Complexity and Non-Linear Scaling

As system functionality increases, FPGA design complexity grows non-linearly rather than linearly. Adding a new module does not only increase code size but also affects:

- shared clock domains
- resource utilization on the FPGA fabric
- signal routing complexity
- timing closure difficulty

For example, integrating multiple systems (counters, PWM control, ultrasonic sensing, LCD display) required careful coordination to ensure that independent modules did not interfere with each other's timing or resource usage. This behavior contrasts with software systems, where modular additions are often more isolated.

7.5 Comparison with Microcontroller-Based Systems

A key observation during this internship is the structural difference between FPGA-based systems and microcontroller-based systems.

Microcontroller Systems:

- sequential execution model
- simpler debugging (software breakpoints, runtime logs)
- limited parallelism
- faster development cycle for simple applications

FPGA Systems:

- true parallel execution at hardware level
- deterministic timing behavior
- high flexibility in architecture design
- significantly higher design and verification complexity

While microcontrollers are more accessible for rapid development, FPGAs provide superior performance for applications requiring real-time parallel processing and hardware-level control.

7.6 Engineering Skills Developed

This internship contributed to the development of several key engineering competencies:

- structured digital system design methodology
- understanding of hardware/software co-design principles
- practical implementation of synchronous sequential logic
- real-world sensor and actuator integration
- systematic debugging and iterative design improvement

Additionally, the experience reinforced an engineering mindset focused on system-level behavior rather than isolated functional components.

7.7 Summary of Technical Insights

The internship demonstrated that FPGA-based embedded systems require a disciplined engineering approach where:

- correctness depends on both logic and timing
- system behavior emerges from hardware structure
- debugging requires hardware validation rather than runtime inspection

Despite the complexity, this methodology enables the development of highly deterministic and efficient real-time systems suitable for advanced embedded applications.

CHAPTER 8: GENERAL CONCLUSION

This internship provided practical exposure to FPGA-based embedded systems and introduced a development methodology fundamentally different from conventional software-oriented approaches.

Throughout the internship, multiple digital and embedded systems were designed and implemented using the Zybo Z7-20 platform. The work progressively evolved from simple programmable logic experiments involving LEDs, switches, and counters toward more advanced integrated systems combining sensors, actuators, displays, and real-time hardware control. The internship also provided direct experience with:

- VHDL-based hardware design
- FPGA synthesis and implementation using Vivado
- Embedded software development on ARM-based processing systems
- Sensor interfacing and actuator control
- Real-time system integration and debugging

One of the most significant outcomes of this experience was the understanding that FPGA development requires a different engineering mindset compared to classical programming. Designing systems at the hardware level involves strict timing constraints, concurrent execution, and structural reasoning rather than sequential software logic. The internship further highlighted the importance of:

- modular system architecture
- rigorous debugging methodology
- synchronization and timing verification
- progressive system integration

From a technical perspective, the experience strengthened practical knowledge in embedded systems engineering and provided a clearer understanding of how hardware and software interact within heterogeneous computing architectures. Finally, this internship represented an important transition from theoretical academic concepts toward practical engineering implementation. It provided valuable insight into the challenges of real-world embedded system development and reinforced the importance of autonomy, experimentation, and iterative problem-solving in engineering practice.